Code559

Front-End Developer Course

MODULE 9

DOM and Events

Find elements, decide what they should do, and connect that to a click.

Free lesson: 559solutions.com/code559/module-09-dom-events/

In this packet

1. **The lesson**
2. **Project: An interests gallery and an FAQ about you**
3. **Finishing work in Git**
4. **Troubleshooting: six common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to find elements on a page, change them, and react to clicks and keys, organizing every script in the same three blocks. This is where everything you've learned becomes visible: the language from Modules 5–8 finally changes what a visitor sees.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 5
- Core concepts 5
  - 1. What does each layer of a page do? 5
  - 2. How do you connect `script.js`, and when does it run? 6
  - 3. What does `querySelector()` give you, and why store it? 7
  - 4. What does `querySelectorAll()` give you, and how do you loop through it? 8
  - 5. How do you change text, and why not `innerHTML`? 9
  - 6. How does `addEventListener()` work, and why no parentheses? 10
  - 7. How do you add, remove and toggle a class? 10
  - 8. How do you show and hide something? 11
  - 9. How do you build page items from an array of objects? 12
  - 10. How is every script in this course organized? 13
  - 11. Why does a real button work with the keyboard for free? 13
- Exercise 14
- Common misconception 18
- Summary 18
- Check your understanding 19
- Before next module 19
- Go further 19
- What to keep 20

Part 2: Project: An interests gallery and an FAQ about you 21

Part 3: Finishing work in Git 34

Part 4: Troubleshooting: six common problems 36

Part 5: Self-assessment 39

Part 6: Answers 41

About 559 Solutions 48

© 2026 559 Solutions LLC

PART 1

The lesson

Find elements, decide what they should do, and connect that to a click.

Time: about 2 to 2½ hours (estimate)

In this module you will learn:

- The three layers of a page: HTML is structure, CSS is appearance, JavaScript is behavior
- How to connect `script.js` to a page, and why it must load after the HTML: bottom of `<body>` or `defer`
- What `querySelector()` gives you: the first match, or `null`
- What `querySelectorAll()` gives you: a collection, and how to loop through it with `for...of`
- Why you store element references in `const` variables
- How to change text with `textContent`, and why this course avoids `innerHTML` for text
- How `addEventListener("click", handler)` works, and why it needs `toggleDetails`, not `toggleDetails()`
- `classList.add()`, `.remove()` and `.toggle()`
- The `hidden` property for showing and hiding
- How to build page items from an array of objects with `createElement()` and `append()`
- The pattern every script in this course follows: find elements → define behavior → connect events
- Why a real `<button>` works with Enter and Space for free

Why this matters

In Modules 5 to 8 you learned the JavaScript language in the Console and in Node. Your code could add, compare, loop and return values. But no visitor ever saw any of it. This module connects that language to a real page. After it, your page can build a list from data, and a click can show a hidden paragraph or change a button's text.

The browser turns your HTML into a tree of objects called the **DOM** (Document Object Model). JavaScript can find any object in that tree, read it and change it. Everything in this module is one of three moves: find an element, change something about it, or wait for the visitor to do something and then change it.

You also learn a habit that keeps scripts readable: every script in this course is organized the same way, in three blocks. When something breaks, you know where to look.

© 2026 559 Solutions LLC

Before you start

1. Finish Module 8. You should be able to write a function, loop over an array with `for...of`, and read a property from an object with a dot (`interest.name`).
2. In your course folder (for example `Documents\code559`), create a throwaway folder called `practice-09`. Nothing in it is published. Open it in VS Code with **File → Open Folder...**.
3. In `practice-09`, make a new file named `icon-star.svg`. Type (or paste) this one line into it and save. It draws the small orange star that the renders below use:

   ```
   <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="2 2 20 20"><polygon points="12,2 15,9 22,9 16.5,13.5 18.5,21 12,16.5 5.5,21 7.5,13.5 2,9 9,9" fill="#fc7614"/></svg>
   ```

   To use your own small icon instead, keep its real name and ending (for example `heart.png`) and change `icon-star.svg` in the script to match. Renaming a PNG to `.svg` does not work: the browser shows a broken image.
4. Keep Edge or Chrome open. You will use **F12 → Console** to read messages and to test lines.

> **Checkpoint.** You have a `practice-09` folder open in VS Code, `icon-star.svg` in it, and you can open the Console in your browser.

Core concepts

1. What does each layer of a page do?

A web page is built from three languages, and each has one job.

© 2026 559 Solutions LLC

HTML is structure: what is on the page (a heading, a button, a paragraph). **CSS is appearance**: how it looks (colors, spacing, layout). **JavaScript is behavior**: what happens when something happens (a click, a key press, a form being sent). Keep the jobs apart. A paragraph that should start hidden is marked in HTML with `hidden`. Its color lives in CSS. The decision to show it lives in JavaScript.



Three files, three jobs. When you are not sure where a line belongs, ask which of the three jobs it does.

For each of these, name the layer: "the button is blue," "there is a button," "clicking the button shows a paragraph."

2. How do you connect `script.js`, and when does it run?

You connect a script with a `<script>` tag and a `src`, the same way you connected a stylesheet with `<link>` in Module 2.

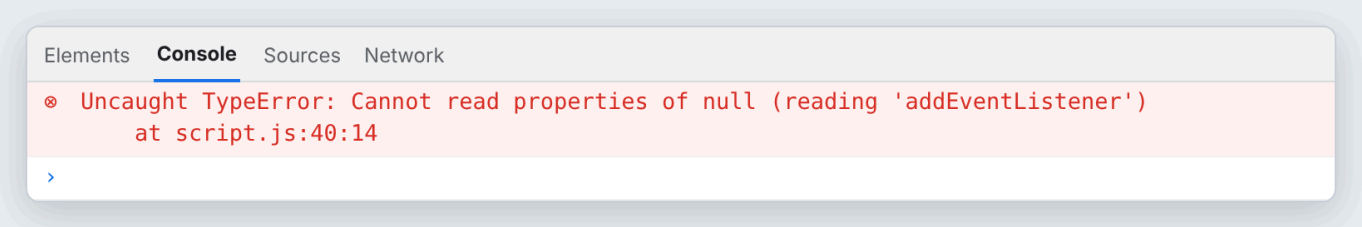
A plain `<script src="script.js">` runs **the moment the browser reaches it**, before it reads the rest of the HTML. If that tag is in the `<head>`, your script runs while the `<body>` does not exist yet. Every `querySelector()` finds nothing and gives back `null`. There are two fixes. You can put the tag at the very bottom of `<body>`, after all the HTML. Or you can keep it in the `<head>` and add `defer`. MDN describes `defer` as meaning "the script is meant to be executed after the document has been parsed, but before firing `DOMContentLoaded`" (MDN, *<script>: The Script element*). This course uses `defer`, so the head holds both the stylesheet and the script:

```
<link rel="stylesheet" href="styles.css">
<script src="script.js" defer></script>
```

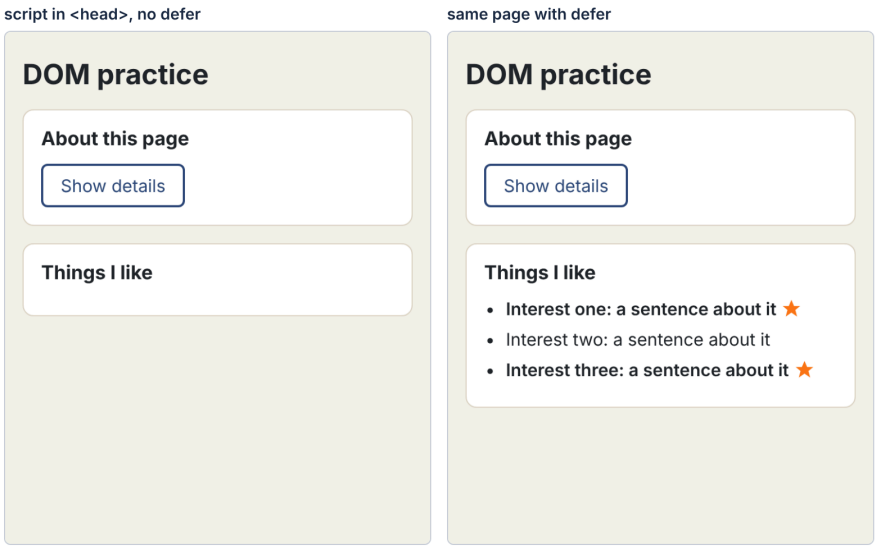
Two notes from the same MDN page: `defer` only works on a script with a `src`, not on code written between the tags, and deferred scripts run in the order they appear.

Here is the exercise page with the script in the `<head>` and **no** `defer`.

© 2026 559 Solutions LLC



`querySelector()` did not fail. It returned `null`, because the button did not exist yet. The error only appears on line 40, the first time the script tries to use that `null`. (real output, rendered; captured in Chromium 141. The message and location come from the page error Chromium reported; line 40 is `toggleButton.addEventListener(…)`.)



Same files, one word different. Without `defer` the script stopped at the error, so the list was never built. With `defer` everything runs. Labels added above each render. (real: rendered in Chromium)

Explain to yourself, out loud, in one sentence, why `null` appears even though the button is in your HTML.[1]

3. What does `querySelector()` give you, and why store it?

`document.querySelector()` takes a CSS selector, the same kind you write in a stylesheet, and finds an element.

MDN: it "returns the first Element within the document that matches the specified CSS selector... If no matches are found, `null` is returned" (MDN, *Document: querySelector() method*). So `"#toggle-btn"` finds the element with that id, `".item"` finds the **first** element with that class, and a typo finds nothing and gives `null`. You store what it finds in a `const`, once, at the top of your script, so every function below can use the name:

```
const toggleButton = document.querySelector("#toggle-btn");
```


© 2026 559 Solutions LLC

The variable does not hold a copy. It holds a **reference** to the real element on the page. Change `toggleButton.textContent` and the page changes.

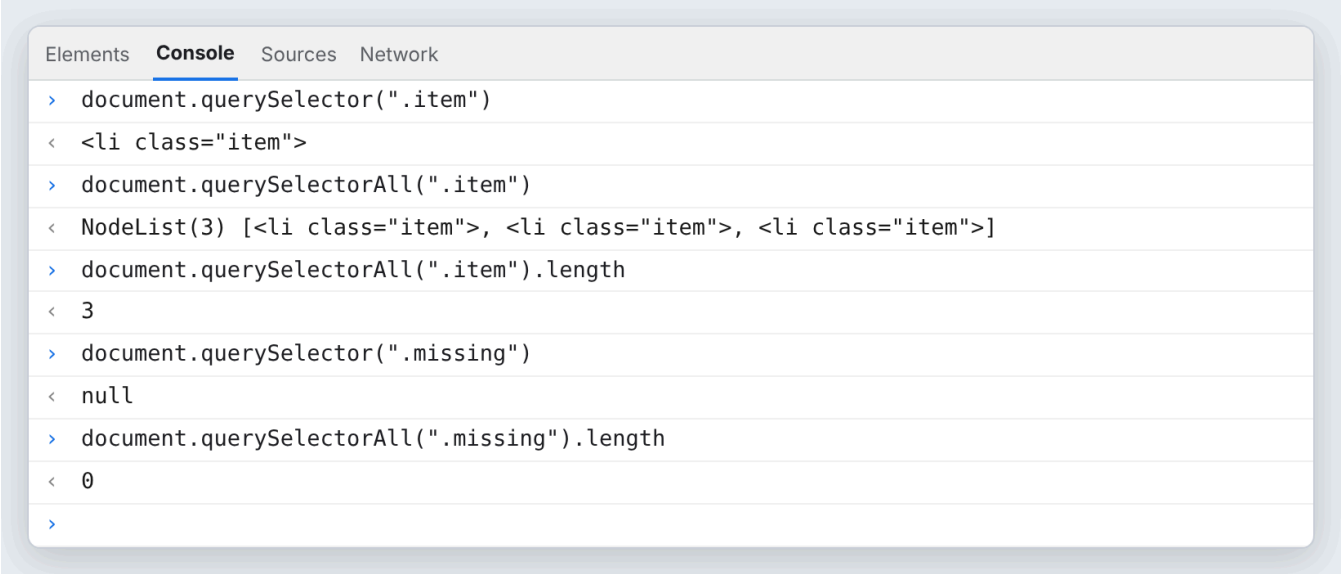
See the first and fourth lines in the image under Concept 4.

Predict what `document.querySelector("#toggle-button")` gives on the exercise page, where the id is `toggle-btn`.

4. What does `querySelectorAll()` give you, and how do you loop through it?

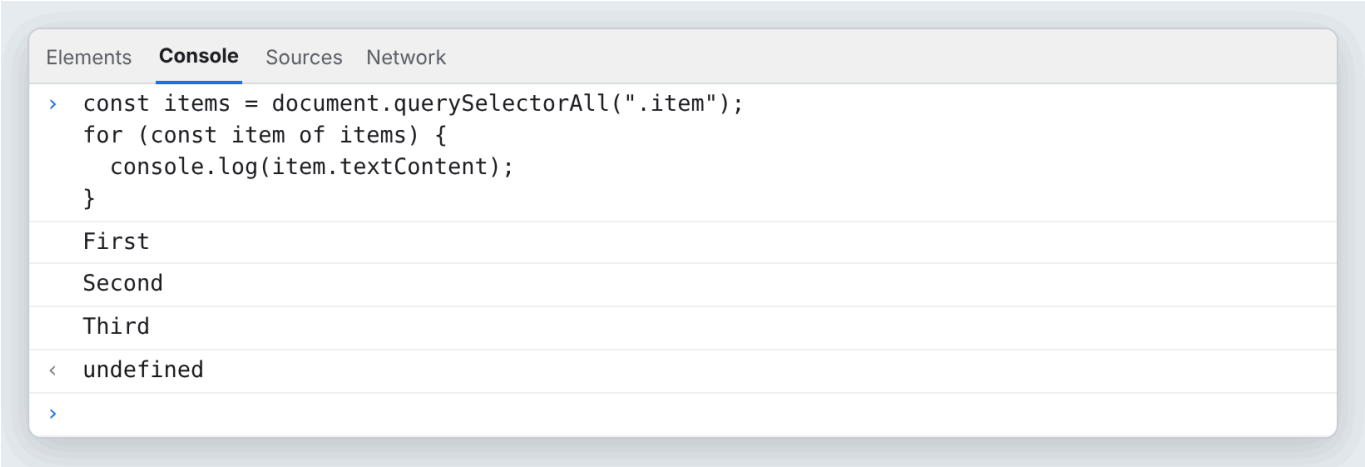
When you want every match, not just the first, use `querySelectorAll()`.

It gives back a **NodeList**, a collection of elements in page order. MDN calls it "a static (not live) NodeList" and adds that when nothing matches you get "an empty NodeList," not `null` (MDN, *Document: querySelectorAll() method*). Static means a snapshot taken at the moment of the call. It does not change later. Checked in Chromium: a list stored in a variable still had a `.length` of 3 after a fourth `li` was added to the page. A new call gave 4. A NodeList has a `.length` and works with the `for...of` loop from Module 8, so you can visit each element in turn. The difference matters: with no match, `querySelector()` gives `null`, and `querySelectorAll()` gives an empty list whose `.length` is `0`.



One element versus a collection, and `null` versus an empty list. (real output, rendered; evaluated in Chromium 141. Real DevTools shows each element as an expandable tag with its text; this frame shows only the opening tag.)

© 2026 559 Solutions LLC



The same `for...of` you used on arrays. Each lap, `item` holds the next element. (real output, rendered; evaluated in Chromium 141)

Predict what `document.querySelectorAll("li").length` gives on the exercise page after the list is built.

5. How do you change text, and why not `innerHTML`?

`textContent` reads or replaces the text inside an element.

Set `toggleButton.textContent = "Hide details";` and the button says exactly that. There is another property, `innerHTML`, that reads its text **as HTML** and builds elements from it. MDN advises against using it for text: "it deals with raw HTML rather than plain text and can be susceptible to XSS attacks" and "it is still less semantic and slower because it needs to invoke the HTML parser" (MDN, *Node: textContent property*). XSS means someone gets their own code to run on your page through text you display. For this course the rule is simple: **text goes in with `textContent`.** New elements are made with `createElement()` (Concept 9).



textContent: Hello

innerHTML: **Hello**

`textContent` shows what you gave it, character for character. `innerHTML` turned the same characters into a real bold element. That is useful for HTML you wrote, and risky for text a visitor typed. (real: rendered in Chromium)

If a visitor types `<b>hi</b>` into a form, which property shows it safely as plain characters?

© 2026 559 Solutions LLC

6. How does `addEventListener()` work, and why no parentheses?

`addEventListener()` tells an element: "when this event happens to you, run this function."

It takes two things: the event name as a string, and the function to run. MDN says the listener "must be `null`, an object with a `handleEvent()` method, or a JavaScript function" (MDN, *EventTarget: addEventListener() method*). That is Module 7's lesson in action. `toggleDetails` is the function itself. `toggleDetails()` runs the function **now** and hands over whatever it returns, which here is `undefined`. So:

```
toggleButton.addEventListener("click", toggleDetails);   // right: hands over the
function
toggleButton.addEventListener("click", toggleDetails()); // wrong: runs it once, now
```

The browser keeps the function and calls it each time the click happens. You do not call it. The click does.

In Module 7 the same mistake with `showMessage()` gave a TypeError. Here you get no error at all. The difference is what the call hands over: `showMessage()` returns a string, which a listener cannot be, and `toggleDetails()` returns `undefined`, which the browser quietly accepts as "no listener." Checked in Chromium 141: `addEventListener("click", undefined)` gave no error, and `addEventListener("click", "Hello")` gave Module 7's TypeError.

You will see the wrong version for real in the Exercise, step 9.

Answer the standing question for this module: **what causes this function to run?** For `toggleDetails`, the answer is "a click on the button," not "the line that mentions it."

7. How do you add, remove and toggle a class?

Every element has a `classList`, a list of its CSS classes that JavaScript can change.

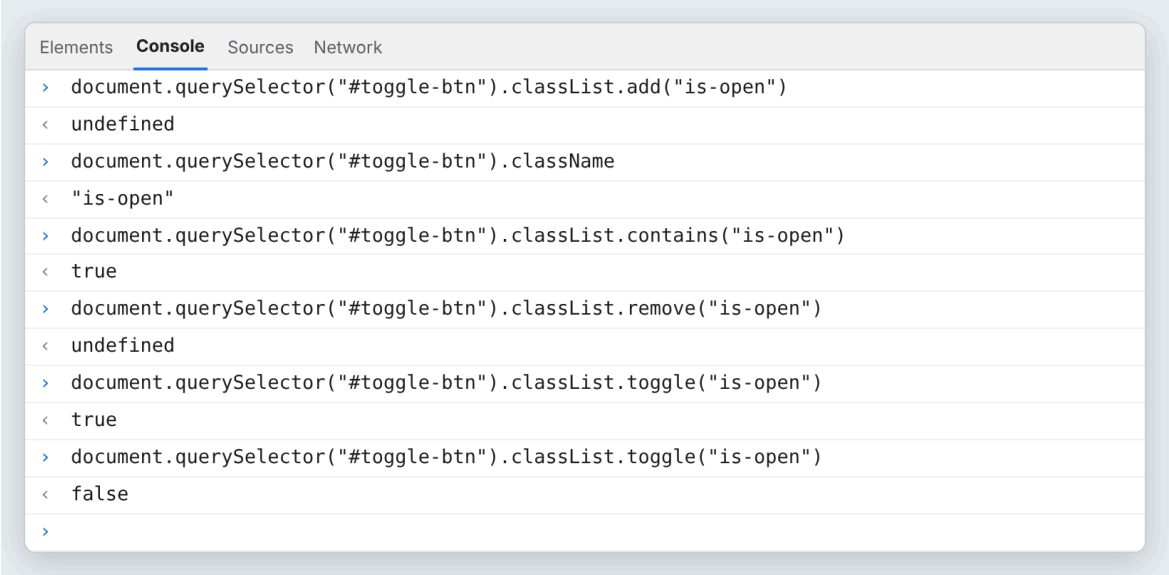
MDN: it "contains a live DOMTokenList collection representing the class attribute of the element," changed with "the `add()`, `remove()`, `replace()`, and `toggle()` methods" (MDN, *Element: classList property*). A DOMTokenList is a list of words, here the class names. Live is the opposite of the static list in Concept 4: it always shows the element's classes as they are now. Checked in Chromium: a `classList` stored in a variable showed the new class after the element's `class` attribute was changed. `add("is-open")` puts the class on. `remove("is-open")` takes it off. `toggle("is-open")`

© 2026 559 Solutions LLC

flips it: on if it was off, off if it was on, and it returns `true` or `false` to tell you which. `contains("is-open")` only asks. This is how JavaScript and CSS share the work: CSS says what `.is-open` looks like, and JavaScript only decides when the class is there.

Typed into the Console for the picture below, in this order. You can type them yourself on the Exercise page once it is open (step 4):

```
document.querySelector("#toggle-btn").classList.add("is-open")
document.querySelector("#toggle-btn").className
document.querySelector("#toggle-btn").classList.contains("is-open")
document.querySelector("#toggle-btn").classList.remove("is-open")
document.querySelector("#toggle-btn").classList.toggle("is-open")
document.querySelector("#toggle-btn").classList.toggle("is-open")
```



`add` and `remove` give back nothing. `toggle` tells you the new state: `true` means the class is now on. (real output, rendered; evaluated in Chromium 141)

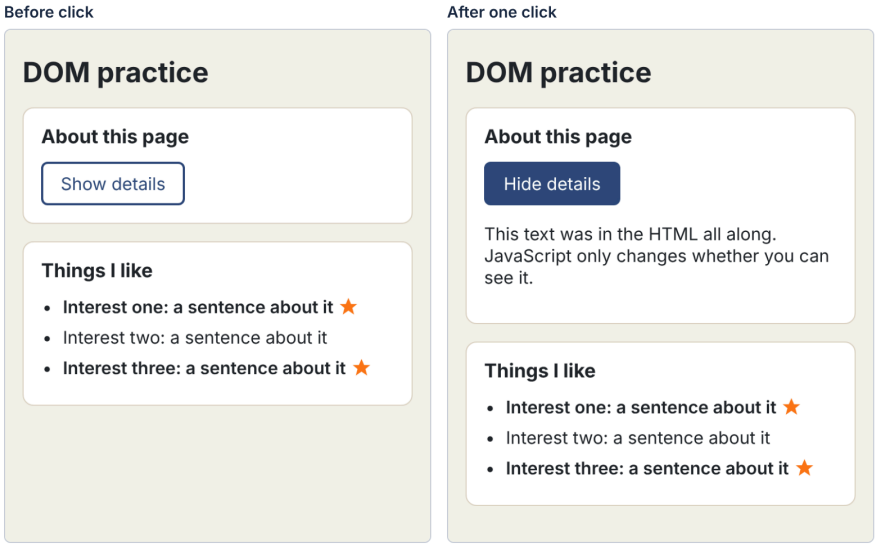
After three `toggle("is-open")` calls in a row on a button that started without the class, is the class on or off?

8. How do you show and hide something?

The `hidden` attribute you met in Module 4 has a matching JavaScript property.

`details.hidden = true` hides it; `details.hidden = false` shows it. To flip it, use `!` from Module 8: `details.hidden = !details.hidden;`. The text stays in the HTML the whole time; only whether it is shown changes. One trap: MDN warns that "changing the value of the CSS `display`

property on a hidden element will override the hidden state" (MDN, *hidden* global attribute). Checked in Chromium: a paragraph with `hidden` and `display: block` in its style had a computed `display` of `"block"`, so it showed. If `hidden` "does nothing," look for a `display` rule on that element.



One click did three things: `hidden` turned off, the text changed with `textContent`, and `classList.toggle("is-open")` made the button dark. Labels added above each render. (real: rendered in Chromium)

Which single line in `toggleDetails` would you delete to make the button change color but never show the paragraph?

9. How do you build page items from an array of objects?

In Module 8 you stored data in an array of objects. Now each object becomes a list item on the page.

`document.createElement("li")` makes a new, empty element. It is not on the page yet. You fill it (with `textContent`, `classList`, `src`, `alt`), and then `list.append(item)` puts it inside the list, after anything already there. MDN: `append()` "inserts a set of Node objects or strings after the last child of the Element" (MDN, *Element: append() method*). A `for...of` loop does this once per object:

```
function buildList() {
  for (const interest of interests) {
    const item = document.createElement("li");
    item.textContent = interest.name + ": " + interest.note;
    if (interest.favorite) {
      item.classList.add("favorite");
```


© 2026 559 Solutions LLC

```
      const star = document.createElement("img");
      star.src = "icon-star.svg";
      star.alt = "favorite";
      item.append(star);
    }
    list.append(item);
  }
}
```

The HTML holds only an empty `<ul id="interest-list"></ul>`. The data decides how many items appear.

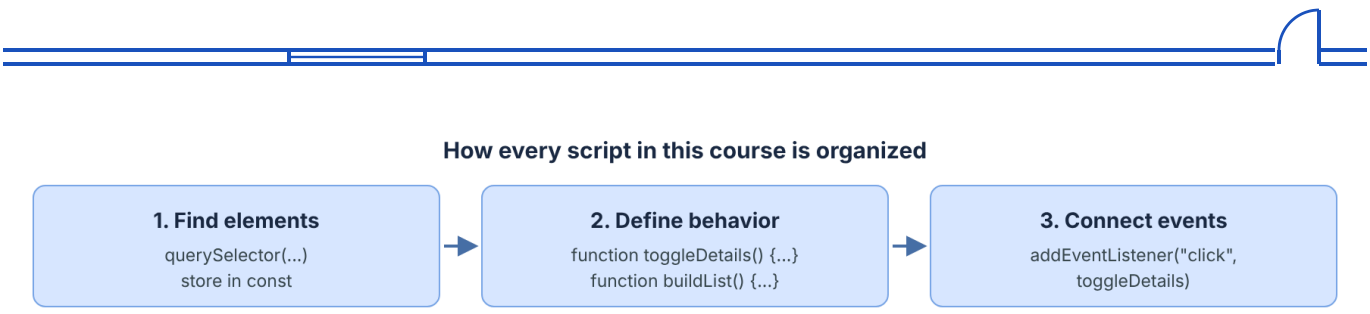
In the right half of the image under Concept 2, three objects became three list items, and the two with `favorite: true` got a bold class and a star.

Predict what changes on the page if you add a fourth object with `favorite: false`.

10. How is every script in this course organized?

Scripts get long. A fixed order makes them easy to read and easy to fix.

Every script has three blocks, marked with comments. **1. Find elements**: every `querySelector()`, stored in `const`, at the top, and the data the script works on, like the `interests` array. **2. Define behavior**: the functions, which say what should happen but do not run yet. **3. Connect events**: the `addEventListener()` lines, plus any function that should run once at load, like `buildList()`. When a button does nothing, check block 3. When you get `null`, check block 1.



Find, define, connect. The Exercise script follows this order exactly.

In the Exercise script, find the comment that starts each block before you type anything else.

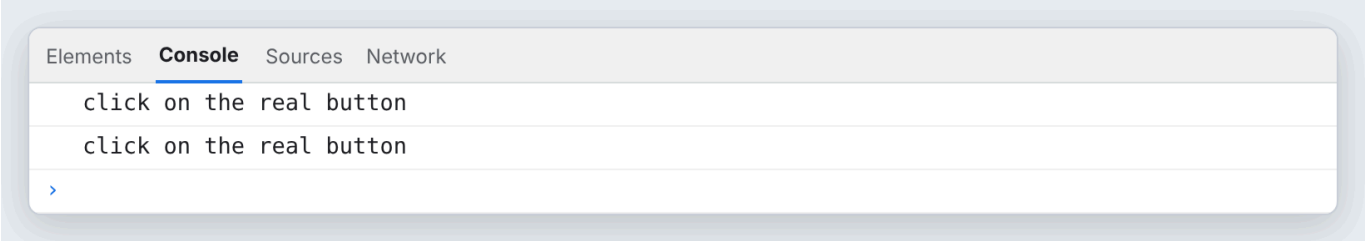
11. Why does a real button work with the keyboard for free?

Some visitors never use a mouse. They move with Tab and press Enter or Space.

© 2026 559 Solutions LLC

A native `<button>` already does this. MDN's button-role page lists Enter and Space as the keys that activate a button and notes that, for native buttons, `onclick` fires for mouse clicks and when the user presses Space or Enter (MDN, *ARIA: button role*). A `<div>` with a click listener does not. Even with `tabindex="0"` so it can take focus, Enter and Space do nothing unless you write extra code. Use `<button type="button">` for anything a visitor clicks that is not a link, and you get focus, keys and a screen-reader name without writing them.

A test page with one real button and one `<div tabindex="0">`, each with a click listener that logs a line. Tab to the button, press Enter, press Space; Tab to the div, press Enter, press Space.



Two key presses on the button gave two clicks. Two key presses on the div gave none. (real output, rendered; key presses sent to Chromium 141 with Playwright)



Tab moved focus to the button (the dark orange ring is the `:focus-visible` style), and Enter ran the same `toggleDetails` a mouse click runs. (real: rendered in Chromium)

You will test this yourself in Exercise step 8.

Exercise

Your hands, about 60 minutes (estimate). Type it, do not paste it. Work in `practice-09`. Nothing here is published.

© 2026 559 Solutions LLC

1. **Create the HTML.** Make `index.html` in `practice-09`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM practice</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main>
    <h1>DOM practice</h1>

    <section class="card">
      <h2>About this page</h2>
      <button id="toggle-btn" type="button">Show details</button>
      <p id="details" hidden>This text was in the HTML all along. JavaScript
only changes whether you can see it.</p>
    </section>

    <section class="card">
      <h2>Things I like</h2>
      <ul id="interest-list"></ul>
    </section>
  </main>
</body>
</html>
```

2. **Add some CSS.** Make `styles.css`. The behavior does not depend on it, so you may style it your own way, but it needs a `button.is-open` rule and a `:focus-visible` rule so you can see the class and the focus. The `#interest-list img` rule keeps the icon 16 pixels square, whatever image you use:

```
body { margin: 0; font-family: "Segoe UI", Arial, sans-serif; background:
#f4f1ea; color: #1f2328; }
main { max-width: 560px; margin: 0 auto; padding: 24px 16px; }
h1 { font-size: 1.6rem; margin: 0 0 16px; }
h2 { font-size: 1.1rem; margin: 0 0 12px; }
.card { background: #fff; border: 1px solid #ddd5c4; border-radius: 10px;
padding: 16px; margin-bottom: 16px; }
button { font: inherit; padding: 8px 16px; border: 2px solid #2d4a7a; border-
radius: 6px; background: #fff; color: #2d4a7a; cursor: pointer; }
button.is-open { background: #2d4a7a; color: #fff; }
button:focus-visible { outline: 3px solid #b45309; outline-offset: 2px; }
```


© 2026 559 Solutions LLC

```
#interest-list { margin: 0; padding-left: 20px; }
#interest-list li { margin-bottom: 8px; }
#interest-list li.favorite { font-weight: 600; }
#interest-list img { width: 16px; height: 16px; margin-left: 6px; vertical-
align: -2px; }
```

3. **Type the script.** Make `script.js`. Type the three comments first, then fill each block. Change the three interests to your own words.

```
// 1. Find elements
const toggleButton = document.querySelector("#toggle-btn");
const details = document.querySelector("#details");
const list = document.querySelector("#interest-list");

const interests = [
  { name: "Interest one", note: "a sentence about it", favorite: true },
  { name: "Interest two", note: "a sentence about it", favorite: false },
  { name: "Interest three", note: "a sentence about it", favorite: true }
];

// 2. Define behavior
function toggleDetails() {
  console.log("toggleDetails ran");
  details.hidden = !details.hidden;
  if (details.hidden) {
    toggleButton.textContent = "Show details";
  } else {
    toggleButton.textContent = "Hide details";
  }
  toggleButton.classList.toggle("is-open");
}

function buildList() {
  for (const interest of interests) {
    const item = document.createElement("li");
    item.textContent = interest.name + ": " + interest.note;
    if (interest.favorite) {
      item.classList.add("favorite");
      const star = document.createElement("img");
      star.src = "icon-star.svg";
      star.alt = "favorite";
      item.append(star);
    }
    list.append(item);
  }
}
```


© 2026 559 Solutions LLC

```
// 3. Connect events
toggleButton.addEventListener("click", toggleDetails);
buildList();
```

4. **Predict, then run.** Before you open the page, write down: how many list items will appear, and which ones get a star? Then open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Check your prediction against the left half of the image under Concept 8.

5. **Click.** Click **Show details** twice. Predict the Console first: how many `toggleDetails ran` lines after two clicks? (Two: one per click.)

6. **Use `querySelectorAll()` in the Console.** Type `document.querySelectorAll("#interest-list li").length`. Then loop over it:

   ```
   for (const item of document.querySelectorAll("#interest-list li")) {
   console.log(item.textContent); }
   ```

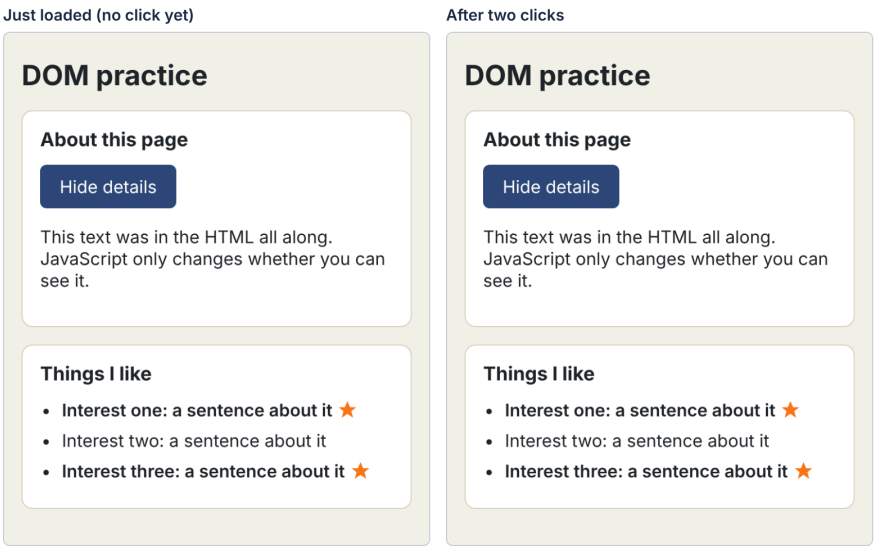
   Does the star image appear in the logged text? (It does not. An `<img>` has no text content; its `alt` is an attribute.)

7. **Modify.** Add a fourth interest object to the array. Save and press **F5**. The page should show four items without any change to the HTML.

8. **Keyboard test.** Click in the address bar, then press **Tab** until the button has a focus ring. Press **Enter**. Press **Space**. Each press should toggle the paragraph and add a line to the Console.

9. **Break it on purpose.** Before you change anything, predict: if you add parentheses in block 3, will the paragraph start hidden? Then add them: `toggleButton.addEventListener("click", toggleDetails());`. Save, press **F5** and click the button twice.

© 2026 559 Solutions LLC



With the parentheses, `toggleDetails()` ran once while the page loaded, so the paragraph opened by itself. The listener received `undefined`, so clicks do nothing. The Console shows exactly one `toggleDetails ran` line and no red error. Labels added above each render. (real: rendered in Chromium)

No error message is the dangerous part. Your clue is the Console: one line at load, none on click. Remove the parentheses, save, and check that clicks work again.

10. **Optional second break.** Delete the word `defer` from the `<script>` tag. Reload and read the red message. Find the line number it names, and explain why `toggleButton` is `null`. Put `defer` back.

Common misconception

> *"My button does nothing, so my function must be wrong."*

What would you say? Hint: a function can be perfect and never run. Check the Console for a `null` error first, then check block 3 for parentheses, then check that the selector in block 1 matches the id in the HTML.

Summary

HTML is structure, CSS is appearance, and JavaScript is behavior. A script in the `<head>` needs `defer`, or it runs before the body exists and every `querySelector()` gives `null`. `querySelector()` gives the first match or `null`; `querySelectorAll()` gives a NodeList, empty when nothing matches, that you loop through with `for...of`. You store each element reference in a `const` at the top of the

© 2026 559 Solutions LLC

script. `textContent` puts in text exactly as written, which is why text never goes in with `innerHTML` in this course. `addEventListener("click", toggleDetails)` hands the browser the function so the click can run it; with parentheses it runs once at load and clicks do nothing. `classList.add()`, `.remove()` and `.toggle()` switch CSS classes on and off. The `hidden` property shows and hides an element, unless a CSS `display` rule overrides it. `createElement()` and `append()` build page items from an array of objects. Every script follows the same order: find elements, define behavior, connect events. A native `<button>` responds to Enter and Space with no extra code; a `<div>` does not.

Check your understanding

1. `document.querySelector(".card")` gives one element, but your page has three cards. Why, and what would you use to get all three?
2. A paragraph has `hidden`, and `details.hidden = true` runs, but the paragraph still shows. Where would you look first?
3. **What causes this function to run?** In the Exercise, `buildList()` and `toggleDetails` are both in block 3. What makes each one run, and how many times?

Before next module

- [ ] Your `practice-09` page builds its list from the array and the button toggles the paragraph
- [ ] You saw the `null` TypeError when `defer` was missing, and fixed it
- [ ] You saw `toggleDetails()` run once at load with no error, and fixed it
- [ ] The button works with Tab, Enter and Space
- [ ] You can say the three blocks of a script in order without looking
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *<script>: The Script element* (the `defer` attribute)
- MDN: *Document: querySelector() method* and *Document: querySelectorAll() method*
- MDN: *Node: textContent property* (the section "Differences from innerHTML")
- MDN: *EventTarget: addEventListener() method*
- MDN: *Element: classList property*; *HTMLElement: hidden property*; *hidden* (global attribute)
- MDN: *Element: append() method*
- MDN: *ARIA: button role* (the keyboard table, and why a native `<button>` is preferred)
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

© 2026 559 Solutions LLC

What to keep

Keep `practice-09` with its three-block `script.js`, and a note of the two breakages you caused: what you changed, what you saw, and what the clue was. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "<script>: The Script element"; MDN "Document: querySelector() method"; MDN "Document: querySelectorAll() method"; MDN "Node: textContent property"; MDN "EventTarget: addEventListener() method"; MDN "Element: classList property"; MDN "HTMLElement: hidden property"; MDN "hidden" (HTML global attribute); MDN "Element: append() method"; MDN "<button>: The Button element"; MDN "ARIA: button role". Checked 2026-10-11: MDN "NodeList" (live and static lists). Every page render, Console result and key-press result was captured in Chromium 141.0.7390.37 (via Playwright) on Linux. Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩

© 2026 559 Solutions LLC

PART 2

Project: An interests gallery and an FAQ about you

Your hands, about 2 hours (estimate). In the Exercise you found elements, changed them, and connected a function to a click in `practice-09`. Now you do the same on your real site, twice. On the Interests page you add a **gallery** that shows one photo at a time, with **Previous** and **Next** buttons that wrap around. On the About page you add an **FAQ about you**: each question is a button that opens and closes its own answer, with a plus or minus icon. You use only what Modules 5 to 9 taught:

- `<script src="…" defer>` in the `<head>`, and `<button type="button">`
- `document.querySelector()` with an id, stored in a `const`
- `textContent`, and the `src` and `alt` properties of an image (as `star.src` and `star.alt` in Core concept 9)
- `addEventListener("click", …)` with a function reference, never a call
- `classList.toggle()`, and the `hidden` property flipped with `!`
- an array of objects read with an index and a dot, and `.length`
- `let` with `++`, and one new sign, `--`
- `if` with `===` and `<`, functions with parameters, and `+` with parentheses (Module 6)
- the three blocks from Core concept 10: find elements, define behavior, connect events

This project starts from the Module 4 packet's site: five pages (Home, About, Interests, Rate, Contact), one shared `styles.css` with the colors and focus rings, and the `images` folder with your interest photos. The scripts from Modules 5 to 8 can stay where they are. No page uses them, and this project does not either.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds two new files, `gallery.js` and `faq.js`, next to `index.html`. `interests.html`, `about.html` and `styles.css` have changed. No other file changes.

One script per page. `interests.html` loads `gallery.js`, and `about.html` loads `faq.js`, each with one `<script src="…" defer>` line in the `<head>`, under the stylesheet link. The other pages load no script. Why two files and not one `script.js`? Block 1 of a script looks for its elements. On a page that does not have them, `querySelector()` gives `null`, and the first `addEventListener()` stops the script with a `TypeError`. So each page gets a script that only looks for what that page has.

The gallery. A new `<section class="gallery">` on the Interests page, after the section with your cards. It holds:

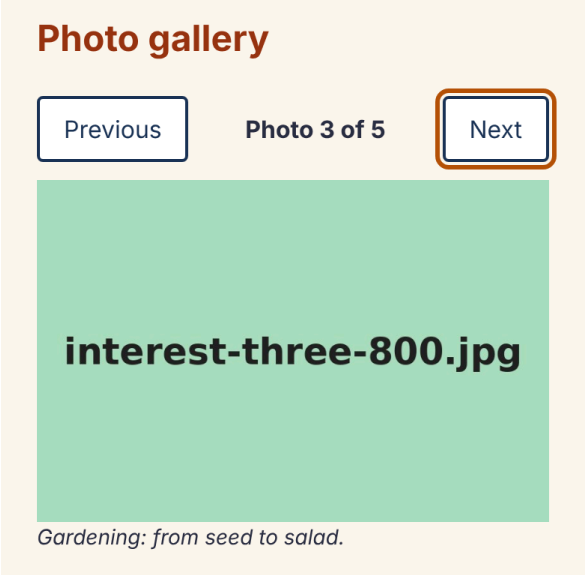
- an `<h2>` that says **Photo gallery**
- a row (`<div class="gallery-controls">`) with a **Previous** button, a counter that says **Photo 1 of 5**, and a **Next** button. The ids are `prev-button`, `photo-count` and `next-button`.

© 2026 559 Solutions LLC

- a `<figure>` with one `<img id="gallery-photo">` and a `<figcaption id="gallery-caption">`

The photos, their alt text and their captions live in `gallery.js`, as an array of objects. Use the 800 px files you made in Module 3, one object per interest. Each object has three properties: `src` (the file, such as `"images/interest-one-800.jpg"`), `alt` (what the photo shows) and `caption` (a short line about the interest).

What the gallery does. Next shows the next photo, with its alt text and caption, and the counter changes to match. Previous goes back. From the last photo, Next goes to the first. From the first photo, Previous goes to the last. The counter always says which photo it is and how many there are, such as **Photo 3 of 5**. The HTML shows the first photo, so the page still makes sense if the script does not run.



The finished gallery on a phone, driven by keyboard only: Tab to Next, then Enter twice. The buttons sit above the photo, so they do not move when a photo of another shape loads. The photos here are placeholders with their file name painted on them, as in Module 3. (real: rendered in Chromium at 390 px)

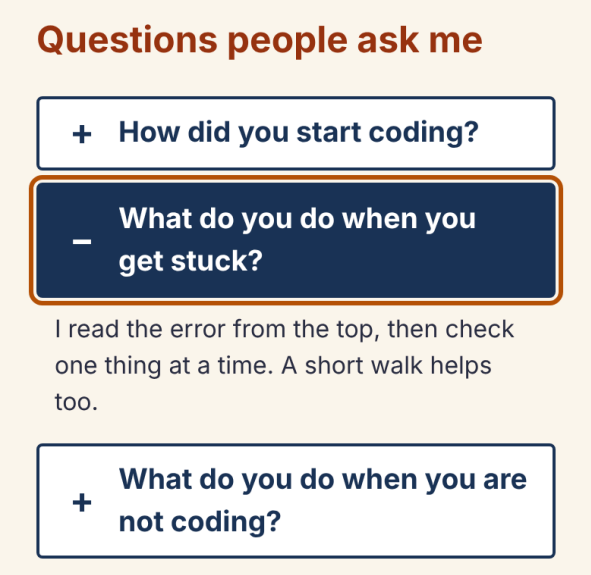
The FAQ. A new `<section class="faq">` on the About page, after your other sections. It holds an `<h2>` that says **Questions people ask me**, then three questions about you. Each question is a `<div class="faq-item">` with two parts:

- An `<h3>` with a `<button class="faq-question" type="button" aria-expanded="false">` inside. The button holds a `<span class="faq-icon">` with a `+`, then the question.
- A `<p class="faq-answer" hidden>` with your answer.

The ids are numbered: `question-1`, `icon-1` and `answer-1` for the first question, then `-2` and `-3`. The heading around each button lets screen-reader users jump from question to question, as they jump between headings.

© 2026 559 Solutions LLC

What the FAQ does. A click on a question shows its answer, changes its `+` to a minus sign, `−`, and turns the button navy with white text. It also tells screen readers that the question is now open. A second click hides the answer and puts it all back. Each question opens and closes only its own answer, and any number can be open at once. With a keyboard, Tab reaches each question, and Enter or Space does exactly what a click does.



The FAQ with the second question opened by keyboard: Tab to it, then Enter. Only its own answer is showing. (real: rendered in Chromium at 390 px)

The colors. Use the custom properties already in your `styles.css`. Every new button has `--ink` text (`#1d3557`) on `--card` (`#ffffff`), 12.4:1. On hover, the background is `--soft` (`#e6f2f0`), 10.8:1. An open question is white on `--ink`, 12.4:1. Every button keeps the dark orange focus ring from Module 4, which is 4.7:1 on your cream page background. These values were checked with the WCAG contrast formula. Nothing scrolls sideways at 390 px.

The ids, classes, file names and button words must match the brief. The photos, captions, questions and answers are yours. Everything on the About page is public, so answer as you would to a stranger: no full name, school or street.

The steps

Type every line. Do not paste. You may copy one finished HTML block to make the next one, once you have typed the first yourself.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.

© 2026 559 Solutions LLC

2. **Plan on paper.** Make two lists. For the gallery: each photo's file name, its alt text and a short caption. For the FAQ: three questions people ask you, and a one- or two-sentence answer to each. Then, for each feature, write the three block headings from Core concept 10 and, under each, what goes there: the elements to find, the functions to define, and the events to connect. Do not write any code yet.

3. **The gallery HTML.** Open `interests.html`. In the `<head>`, under the stylesheet link, add:

   ```
   <script src="gallery.js" defer></script>
   ```

   Then, after the closing `</section>` of your cards and before `</main>`, add the new section. Use your first photo's file, alt text and caption:

   ```
   <section class="gallery">
     <h2>Photo gallery</h2>
     <div class="gallery-controls">
       <button id="prev-button" type="button">Previous</button>
       <p id="photo-count">Photo 1 of 5</p>
       <button id="next-button" type="button">Next</button>
     </div>
     <figure>
       <img id="gallery-photo" src="images/interest-one-800.jpg"
            width="1200" height="800" alt="A trail through the hills">
       <figcaption id="gallery-caption">Hiking: the view at the top is worth the
   climb.</figcaption>
     </figure>
   </section>
   ```

   Set `width` and `height` to the real size of your 1200 file, as in Module 3. The image has no `srcset` on purpose: Problem 3 in Part 4, *Troubleshooting*, shows what `srcset` does to a gallery.

   Predict before you run: `gallery.js` does not exist yet. What will the page show, and what will the Console say? Save. Open `interests.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. The sample page showed the first photo under the two buttons, and the buttons did nothing. The Console had one red line: `Failed to load resource: net::ERR_FILE_NOT_FOUND`, with `gallery.js` beside it. The HTML works alone. The behavior is missing.

© 2026 559 Solutions LLC

4. **The gallery CSS.** Open `styles.css`. Above the `/* Wider screens: 700px and up */` comment, add a new group:

```
/* Interests page: gallery */
.gallery {
  max-width: 640px;
}

.gallery-controls {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 8px;
  margin-bottom: 12px;
}

.gallery-controls p {
  margin: 0;
  font-weight: 700;
}

.gallery-controls button,
.faq-question {
  padding: 8px 16px;
  font: inherit;
  color: var(--ink);
  background-color: var(--card);
  border: 2px solid var(--ink);
  border-radius: 4px;
  cursor: pointer;
}

/* Continue alone: one rule for .gallery-controls button:hover
   and .faq-question:hover that sets background-color to var(--soft). */
```

   The FAQ buttons share this look, so their class is in the same rule now. Save and press F5. The two buttons are white with a navy border, and the counter sits between them. Press Tab until Previous has a dark orange ring. That ring comes from your Module 4 rule for `button:focus-visible`, so you did not write it again.

5. **`gallery.js`, block 1: find elements.** In the Explorer, create `gallery.js` next to `interests.html`. Type the first line and the first block:

```
// gallery.js: one photo at a time on the Interests page, with Previous and Next

// 1. Find elements
```


© 2026 559 Solutions LLC

```
const photo = document.querySelector("#gallery-photo");
const caption = document.querySelector("#gallery-caption");
const counter = document.querySelector("#photo-count");
const previousButton = document.querySelector("#prev-button");
const nextButton = document.querySelector("#next-button");

const photos = [
  {
    src: "images/interest-one-800.jpg",
    alt: "A trail through the hills",
    caption: "Hiking: the view at the top is worth the climb."
  },
  // Continue alone: one object for each of your other photos,
  // with the same three properties in the same order.
];

let current = 0;
```

Block 1 holds the five elements, the data, and one more thing: `current`, the index of the photo on screen. It starts at 0, the first photo. It is a `let`, because every click changes it. Add your other photos, then delete the two "Continue alone" lines. Save and press F5. The red line about `gallery.js` is gone, and nothing else changed: block 1 only finds and stores.

Now test block 1 in the Console. Type `photos.length`, then `photos[1].alt`, then `current`. The sample gave `5`, the text `A pot of soup on the stove`, and `0`. Then type `photo` and press Enter. The Console showed the `<img>` element itself: `photo` holds a reference to the real element (Core concept 3).

6. **Block 2: `showPhoto`.** Under block 1, leave one blank line and type:

```
// 2. Define behavior
function showPhoto() {
  const item = photos[current];
  photo.src = item.src;
  photo.alt = item.alt;
  caption.textContent = item.caption;
  counter.textContent = "Photo " + (current + 1) + " of " + photos.length;
}
```

Read it in two parts. First, `item` is the object at position `current`. Then four lines copy its facts onto the page: the file and the alt text onto the image, and the caption and the counter as text. The counter adds 1 because people count from 1 and arrays count from 0. The parentheses make JavaScript add first and join after, as `"Total: " + (2 + 3)` did in Module 6.

© 2026 559 Solutions LLC

Nothing runs yet: a function only says what to do. Test it by hand. Save, press F5, and type `current = 2` in the Console, then `showPhoto()`. The sample showed your third photo, its caption, and **Photo 3 of 5**. The Console printed `undefined` under `showPhoto()`, because the function returns nothing (Module 7).

7. **`showNext` and `showPrevious`: wrap around.** Under `showPhoto`, leave one blank line and type:

```
function showNext() {
  current++;
  if (current === photos.length) {
    current = 0;
  }
  showPhoto();
}
```

`current++` moves to the next index. With 5 photos the last index is 4, so after photo 5 `current` becomes 5, and there is no photo 5 in the array. That is the moment to wrap: the `if` sends `current` back to 0. Then `showPhoto()` puts the new photo on the page. Calling one of your own functions from inside another works like the summary in Module 8, which used your functions together.

Now write `showPrevious` alone, under `showNext`. It has the same shape, the other way round. `current--` is the new sign: the opposite of `++`, it takes 1 away. When `current` drops below 0, it wraps to the last index, `photos.length - 1` (Module 8). Then it calls `showPhoto()`.

```
function showPrevious() {
  // Continue alone: subtract 1 from current; if it is now below 0,
  // set it to the last index; then show the photo.
}
```

Write the three lines, then delete the comment lines, so your line numbers match the sample's. Save, press F5, and test both by hand in the Console: `current = 4`, then `showNext()`, then `showPrevious()`. The sample showed **Photo 1 of 5**, then **Photo 5 of 5**.

8. **Block 3: connect events.** Under the functions, leave one blank line and type:

```
// 3. Connect events
previousButton.addEventListener("click", showPrevious);
nextButton.addEventListener("click", showNext);
showPhoto();
```

Think this through: what causes each function to run? `showPrevious` and `showNext` run on a click, as many times as there are clicks. `showPhoto()` has parentheses, so it runs once, now, as the page loads, like `buildList()` in the Exercise. It makes the counter match your array. If you

© 2026 559 Solutions LLC

have four photos, the HTML still says **of 5**, and this one call corrects it.

Save and press F5. Test with the mouse: click **Next** until you are back at **Photo 1**, counting the clicks. Then click **Previous** once from photo 1. The sample needed five clicks to get round, and Previous went from **Photo 1 of 5** to **Photo 5 of 5**. Then test with the keyboard: press F5, put the mouse aside, and press Tab until Next has the ring. Press **Enter**, then **Space**. Each press moves one photo, the same as a click. You wrote no code for keys: a real `<button>` gives you that (Core concept 11).

9. **Modify.** Make two changes, one at a time. Predict each before you save, then undo it.
   - **No parentheses.** In `showPhoto`, change the counter line to `"Photo " + current + 1 + " of " + photos.length`. What will the counter say at load, and after one click on Next? Write it down, then save and press F5. The sample said **Photo 01 of 5**, then **Photo 11 of 5**. Without the parentheses, JavaScript works from left to right: `"Photo " + current` is already text, so the `1` is joined on as a character, not added. Put the parentheses back.
   - **Another starting photo.** Change `let current = 0;` to `let current = 4;`. The HTML still holds photo 1. Which photo will the page show after F5? Save and look. The sample showed **Photo 5 of 5** with the fifth photo: block 3's `showPhoto()` ran at load and replaced what the HTML had. One click on Next went to **Photo 1 of 5**. Change it back to `0`.
10. **Break it on purpose.** In block 3, add parentheses after `showNext`:

    ```
    nextButton.addEventListener("click", showNext());
    ```

    Predict: which photo will the page show after F5, and what will a click on Next do? Write it down. Then save, press F5, click **Next** twice and **Previous** once.

    The sample opened on **Photo 2 of 5**, with no click at all. Next did nothing, and Previous went to **Photo 1 of 5**. The Console had no red line. Here is why. With the parentheses, `showNext()` ran once at load, which moved `current` to 1. Then `showPhoto()` at the end showed photo 2. The listener got what `showNext()` returned, `undefined`, so a click on Next runs nothing (Core concept 6). Previous still works, because its line has no parentheses. Remove them, save, and test Next again.
11. **The FAQ HTML.** Open `about.html`. In the `<head>`, under the stylesheet link, add:

    ```
    <script src="faq.js" defer></script>
    ```


© 2026 559 Solutions LLC

Then, after your last section and before `</main>`, add the new section with your first question and answer:

```
<section class="faq">
  <h2>Questions people ask me</h2>
  <div class="faq-item">
    <h3>
      <button id="question-1" class="faq-question" type="button" aria-
expanded="false">
        <span id="icon-1" class="faq-icon">+</span>
        How did you start coding?
      </button>
    </h3>
    <p id="answer-1" class="faq-answer" hidden>In 2025, with one hour a day,
most days. My first page went live this year.</p>
  </div>
  <!-- Continue alone: two more faq-item blocks, the same shape,
       with the ids question-2, icon-2, answer-2 and question-3,
       icon-3, answer-3. -->
</section>
```

Add your other two questions, then delete the comment lines. Every answer starts with `hidden`: the page opens with all three closed. Save, open `about.html` in Edge or Chrome, and press F5. The three questions show as plain buttons, no answer shows, and the Console has the same red `ERR_FILE_NOT_FOUND` line as in step 3, this time for `faq.js`.

12. **The FAQ CSS.** In `styles.css`, under the gallery group and above the wider-screens comment, add:

```
/* About page: FAQ */
.faq {
  max-width: 640px;
}

.faq h3 {
  margin: 0 0 8px;
}

.faq-question {
  width: 100%;
  display: flex;
  align-items: center;
  gap: 12px;
  text-align: left;
  font-weight: 700;
}
```


© 2026 559 Solutions LLC

```
.faq-question.is-open {
  color: white;
  background-color: var(--ink);
}

/* Continue alone: .faq-icon has a min-width of 24px, a font-size
   of 24px, a line-height of 1, and centered text. .faq-answer has
   margin 0 0 16px and padding 0 12px. */
```

The border, padding and colors come from the shared rule in step 4. `.faq-question.is-open` has no space between the two classes: it means "a button that has both classes." JavaScript adds `is-open`, and this rule says what it looks like (Core concept 7). The `min-width` keeps every icon the same width, so the questions line up even when one wraps onto two lines. Do not give `.faq-answer` a `display` rule: it would override `hidden` (Core concept 8, and Problem 5 in Part 4).

13. **`faq.js`, block 1.** Create `faq.js` next to `about.html`:

```
// faq.js: questions about me on the About page; each button opens and closes
its own answer

// 1. Find elements
const question1 = document.querySelector("#question-1");
const answer1 = document.querySelector("#answer-1");
const icon1 = document.querySelector("#icon-1");

// Continue alone: the same three lines for question 2,
// then for question 3, each group after a blank line.
```

Nine names in all, three for each question. Type the other six, then delete the comment lines. Save, press F5 and test in the Console: `answer2.hidden` gave `true` in the sample, and `icon3.textContent` gave the text `+`. If one of them gives `null` or an error, compare its selector with the id in `about.html`, character by character.

14. **Block 2: one function for any question.** Under block 1, leave one blank line and type:

```
// 2. Define behavior
function toggleAnswer(question, answer, icon) {
  answer.hidden = !answer.hidden;
  if (answer.hidden) {
    icon.textContent = "+";
  } else {
    icon.textContent = "−";
  }
```


© 2026 559 Solutions LLC

```
  question.classList.toggle("is-open");
  question.ariaExpanded = !answer.hidden;
}
```

This is `toggleDetails` from the Exercise with three parameters. It does not know which question it works on. It works on whichever three elements you hand it. So one function serves all three questions.

The `–` is a real minus sign, not the hyphen key: a hyphen looks shorter than the `+`. It is the one character you may copy from this page and paste. The last line tells screen readers whether the answer is open. `ariaExpanded` is a property, like `hidden`, and it sets the button's `aria-expanded` attribute to `true` or `false`. Chromium then reports the button as expanded or collapsed. The HTML sets the starting value, `aria-expanded="false"`, to match the hidden answer.

But a listener cannot hand it anything. `addEventListener` takes a function **reference**, and a reference has no parentheses, so it has no place for arguments. Writing `toggleAnswer(question1, answer1, icon1)` in block 3 would run it once at load, the mistake from step 10. The fix is a small function for each question, with no parameters, whose only job is to make the call:

```
function toggleFirst() {
  toggleAnswer(question1, answer1, icon1);
}

// Continue alone: toggleSecond and toggleThird, the same shape.
```

Write the two functions, then delete the comment line. Save, press F5 and test by hand: type `toggleFirst()` in the Console. The first answer opened, its icon became `–`, and the button turned navy. Type `question1.ariaExpanded`: the sample gave the text `true`. Type `toggleFirst()` again: all closed.

15. **Block 3, and the full test.** Under the functions, type:

```
// 3. Connect events
question1.addEventListener("click", toggleFirst);
// Continue alone: connect question2 to toggleSecond
// and question3 to toggleThird.
```

Connect the other two, then delete the comment lines. Save and press F5. With the mouse: click each question once, then each again. Each one opens and closes only its own answer. Open two at once: both stay open. With the keyboard: press F5, put the mouse aside, and press Tab. In the

© 2026 559 Solutions LLC

sample, Tab went through the five nav links, then the three questions in order. On each question, press **Enter**, then **Space**. Each press opens or closes it, with the same icon and color change as a click.

Modify. Delete the word `hidden` from `answer-1` in the HTML. Predict what the page shows at load, then after one click, then after a second click. In the sample, the answer showed at load next to a `+` on a white button. The first click hid it and turned the button navy. The second click showed it with a `–`. The icon and the color were the wrong way round, because `toggleAnswer` flips the class and the answer separately, and they started out of step. The HTML and the CSS must start in the same state. Put `hidden` back.

16. **Debug: two questions that do nothing.** Here is a typing mistake that is easy to make. In block 1 of `faq.js`, change `"#question-2"` to `"#question2"` (drop the dash). Save and press F5. Click all three questions.

    Before you read on, write down what you see, and which block you would check first. Then look at the Console.

    In the sample, question 1 worked, and questions 2 and 3 did nothing. The Console showed one red message:

    ```
    Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
        at faq.js:42:11
    ```

    Follow the clues in order. Line 42 is `question2.addEventListener(…)`, in block 3. The message says the thing before `.addEventListener` is `null`, so `question2` holds `null`. A name gets its value in block 1, and there, `querySelector("#question2")` found no element with that id: the HTML says `question-2`.

    The script stopped at line 42, so line 43 never connected question 3 either. Line 41 had already run, which is why question 1 still worked. Your line numbers match the sample's if your `faq.js` has the same shape. Put the dash back, save, and test all three.

17. **Explain it.** Read `gallery.js` to yourself, out loud, from top to bottom. For each name in block 1, say what it holds. For each function, say what causes it to run, and how many times. Then do the same for `faq.js`, and say why `toggleFirst` exists at all. If there is a line you cannot explain, reread the Core concept it comes from.

18. **Test the site.** Tick each box in a full-width window and at 390 px in device mode (F12, then Ctrl+Shift+M). Keep the Console open the whole time.

    - [ ] No red line in the Console on any of the five pages
    - [ ] Next goes through every photo and back to photo 1; Previous from photo 1 goes to the last photo
    - [ ] The photo, its alt text, the caption and the counter always match

© 2026 559 Solutions LLC

- [ ] Each question opens and closes only its own answer, and the icon, the color and `aria-expanded` always match the answer
- [ ] Tab reaches Previous, Next and every question, each with a dark orange ring; Enter and Space work on each one
- [ ] Nothing scrolls sideways at 390 px on the Interests page or the About page
- [ ] The parentheses from step 10 and the typo from step 16 are gone, and every answer has `hidden` again
- [ ] Nothing in your questions, answers or captions is private

19. **Save it in Git.** Go to Part 3, *Finishing work in Git*, and publish both features with the save routine.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

The gallery and the FAQ work on your computer. Now you put them online with the same save routine as in Modules 1 to 8: `git status`, `git add .`, `git commit`, `git push`, then check the live site. Two things are new this time. Your pages now run scripts, so you read the Console before you commit and again on the live site. And a browser may keep an old copy of a script, so you learn to reload without it.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 8.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **Read the Console before you commit (NEW).** A page can look right and still have a broken script. Open each of your five pages in Edge or Chrome with the Console open (**F12**, then the **Console** tab), and press **F5** on each. There should be no red line on any of them. Then click **Next** at least once. The mistake from Project step 10 prints no red line at all, so only a click shows it. Commit only pages whose test from Project step 18 you have just run and seen pass.

3. **See what changed.** Type:

   ```
   git status
   ```

   Under **Changes not staged for commit** you should see `modified:` for `about.html`, `interests.html` and `styles.css`. Under **Untracked files** you should see `faq.js` and `gallery.js`. If `index.html`, `rate.html` or `contact.html` shows up, you changed a page this project does not touch. Open it and check what changed before you go on.

4. **Stage the changes.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** you should now see five lines: `modified:` for the two pages and `styles.css`, and `new file:` for the two scripts.

5. **Commit.** Type:

   ```
   git commit -m "Add an interests gallery and an FAQ about me"
   ```

   On the sample site, Git printed `5 files changed, 225 insertions(+)` on the second line, then `create mode 100644 faq.js` and `create mode 100644 gallery.js`. Your commit id and line count will differ. The two features share the button rule in `styles.css`, so they go into one

© 2026 559 Solutions LLC

commit.

6. **Push.** Type:

   ```
   git push
   ```

   The last line ends in `main -> main`.

7. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 8 commit (or your latest one) under it.

8. **Check the live pages with the Console open (NEW).** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/interests.html` with the Console open, and press F5. There should be no red line. Go through the gallery both ways, past both ends. Then open the live About page and open and close each question.

9. **Reload without the saved copy (NEW).** Browsers keep copies of files they have loaded, to open pages faster. So after a push, a browser may show the new page with an old script, or an old page for a while.

   If the live page acts like an older version, reload it while ignoring that saved copy: press **Ctrl+Shift+R**. Chrome's and Edge's keyboard shortcut pages both list it as reloading the current page "ignoring cached content" (Chrome Help, *Chrome keyboard shortcuts*; Microsoft Support, *Keyboard shortcuts in Microsoft Edge*).

   The steps in this packet were tested with files opened from disk, not on a live site, so watch for this the first time you check.

10. **Test the live pages with the keyboard.** Put the mouse aside. On the live Interests page, Tab to **Next** and press Enter and Space. On the live About page, Tab to each question and press Enter, then Space. Each press does what a click does, with a dark orange ring on the button that has focus.

11. **Test on a phone.** If you have a phone, open your live Interests page on it. Tap **Next** until the gallery wraps, then open each question on your About page. Nothing should scroll sideways.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. On the live site, the Console shows no red line on the Interests or About page, the gallery wraps both ways, and each question opens and closes its own answer with a mouse and with the keyboard.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: six common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose in the sample site and run in Chromium 141, with the files opened from disk as in the steps. The Console messages are quoted as Chromium printed them. The line numbers are the sample's: yours match only if your file has the same shape. After each fix, save, press **F5**, and run the test from Project step 18 again.

1. TypeError: Cannot read properties of null

Symptom. A red message in the Console, and some or all buttons do nothing:

```
Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
    at faq.js:42:11
```

Why. The name before `.addEventListener` holds `null`, because `querySelector()` found no element (Core concept 3). The script stops at that line, so every listener below it is never connected. Listeners above it still work. The sample got this message four ways:

What was wrong	Example	Where the sample stopped
A selector that does not match the id	`"#question2"` for `id="question-2"`	`faq.js:42:11`; question 1 still worked
The `#` missing from the selector	`"next-button"`	`gallery.js:67:12`; Previous still worked
`defer` missing from a script in the `<head>`	`<script src="gallery.js">`	`gallery.js:66:16`, the first listener
The wrong script on the page	`about.html` loading `gallery.js`	`gallery.js:66:16`, the first listener

Fix. Go to the line the message names, and find the name just before `.addEventListener`. Then go to block 1, find where that name gets its value, and compare the selector with the id in the HTML, character by character. If the message points at your **first** listener line, every element may be `null`. Then check the `<script>` tag: is `defer` there, and is it the right file for this page?

2. Failed to load resource: net::ERR_FILE_NOT_FOUND

Symptom. A red line in the Console with a file name beside it. If the file is a script, no button on the page works, and there is no `TypeError`. If it is a photo, the gallery shows a small broken-image icon and the alt text for that photo only, and the red line appears when you reach it.

© 2026 559 Solutions LLC

Why. The browser asked for a file that is not there. With `<script src="galery.js" defer>`, the sample printed this line with `galery.js` beside it, and none of the gallery code ran. With `images/intrest-two-800.jpg` in the `photos` array, the line appeared on the first click on Next, with `intrest-two-800.jpg` beside it. You also see this line before you create the file, as in Project steps 3 and 11.

Fix. Compare the name beside the red line with the real file name in the VS Code Explorer: spelling, dashes, the folder (`images/`) and the ending (`.js`, `.jpg`). Fix the name in the HTML or in the array, not the file, unless the file itself has the typo.

3. The caption and the counter change, but the photo does not

Symptom. Next changes the caption and says **Photo 2 of 5**, but the first photo stays on screen. No red line.

Why. The gallery's `<img>` has a `srcset`, often because it was copied from one of the cards. When an image has a `srcset`, the browser picks its file from that list. In the sample, Chromium showed `interest-one-400.jpg` from the list at load, and kept showing it after `photo.src` changed to the second photo's file. The list still named only the first photo's files.

Fix. Remove `srcset` and `sizes` from the gallery's `<img>`, as in the brief. The gallery shows the 800 px files, which suit both a phone and the 640 px gallery on a laptop.

4. TypeError: Cannot read properties of undefined (reading 'src')

Symptom. The gallery works until it should wrap. Then one click gives this red message, and nothing changes on the page:

```
Uncaught TypeError: Cannot read properties of undefined (reading 'src')
    at showPhoto (gallery.js:43:20)
```

The next click does wrap. So the gallery needs two clicks to go round instead of one.

Why. `current` reached an index with no photo, and `photos[current]` gave `undefined` (Module 8). Reading `.src` from `undefined` is not allowed. The sample got this two ways. With `if (current > photos.length)` in `showNext`, `current` became 5 on photo 5 and was not reset, because 5 is not greater than 5. With `current = photos.length;` in `showPrevious`, it became 5 when wrapping back from photo 1.

Fix. Wrap exactly at the edges. In `showNext`, reset when `current === photos.length`. In `showPrevious`, the last photo is at `photos.length - 1`, because counting starts at 0.

© 2026 559 Solutions LLC

5. An answer that shows all the time, or an icon that is the wrong way round

Symptom. Every answer is visible when the page loads. A click changes the icon and the color, but the answer never hides. Or: the answer hides and shows, but a closed question is navy with a `+`, and an open one is white with a `−`.

Why. In the first case, a `display` rule on the answers overrides `hidden` (Core concept 8). With `display: block;` added to `.faq-answer`, the sample showed all three answers at load, and `hidden` became `true` and `false` with no change on screen. In the second case, the answer did not start with `hidden` in the HTML. `toggleAnswer` flips the answer and the class separately, so they stay out of step (Project step 15).

Fix. Take any `display` rule off `.faq-answer`. Check that every answer in `about.html` starts with `hidden`, and every button starts without `is-open` and with `aria-expanded="false"`.

6. TypeError: Assignment to constant variable.

Symptom. The page loads with photo 1 and the right counter, but Next and Previous do nothing. Each click adds a red message:

```
Uncaught TypeError: Assignment to constant variable.
    at showNext (gallery.js:50:10)
```

Why. `current` was made with `const current = 0;`. A `const` cannot get a new value (Module 6), and `current++` gives it one. The error comes on the first click, not at load: `showPhoto()` only **reads** `current`, so load works. On Previous, the message names `showPrevious` and the `current--` line instead.

Fix. Use `let current = 0;`. Everything else in block 1 stays a `const`: the elements and the array never change, only which photo is showing.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 10. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can name what HTML, CSS and JavaScript each do on a page, and put a new line in the right file	☐	☐	☐	☐
I can load a script with defer, and explain the null error when a script runs before the body exists	☐	☐	☐	☐
I can find an element with querySelector(), store it in a const, and fix a selector that gives null	☐	☐	☐	☐
I can loop through a querySelectorAll() collection with for...of, and say how it differs from one match	☐	☐	☐	☐
I can change text with textContent, and explain why text never goes in with innerHTML in this course	☐	☐	☐	☐
I can connect a function with addEventListener(), and spot the parentheses that run it once at load	☐	☐	☐	☐
I can show and hide with hidden and switch a class with classList, and keep them in step	☐	☐	☐	☐
I can build page items from an array of objects with createElement() and append()	☐	☐	☐	☐
I can organize a script as find, define, connect, and test every button with a mouse and with Tab, Enter and Space	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 10, `showNext()` with parentheses made the gallery open on photo 2, with no red line. In Project step 16, one wrong selector gave a red `TypeError`, and two questions stopped working. Explain why one mistake gives an error and the other does not, and what the clue was in each case.

© 2026 559 Solutions LLC

2. Pick one prediction from the Exercise or the project where your first answer was wrong. What did you write down, what did the page or the Console show, and what do you understand now that you did not before?

3. A friend wants one function for all three FAQ questions, so they wrote block 3 like this. What does the About page show right after F5? What happens on each click on the first question? And what would you tell them to change?

```
// 3. Connect events
question1.addEventListener("click", toggleAnswer(question1, answer1, icon1));
question2.addEventListener("click", toggleSecond);
question3.addEventListener("click", toggleThird);
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **One card instead of three.** `querySelector()` gives the **first** element that matches, and stops looking. The other two cards match too, but it never gets to them. To get all three, use `document.querySelectorAll(".card")`. It gives a NodeList with all three in page order, and you visit each one with a `for...of` loop. (Core concepts 3 and 4)
2. **`hidden` is `true`, but the paragraph still shows.** Look in the CSS first, for a `display` rule that reaches that paragraph. A rule such as `display: block` overrides the `hidden` state, so the paragraph shows whatever the property says. Take the `display` rule off that element, or make it reach other elements only. (Core concept 8; Problem 5 in Part 4)
3. **What causes this function to run?** `buildList()` has parentheses, so the line in block 3 calls it. It runs once, while the page loads, and builds the list. `toggleDetails` has no parentheses. The line only hands the function to the browser, and nothing runs at load. A click runs it, once per click, and so do Enter and Space on the focused button. So it runs zero times at load, then once each time the visitor clicks or presses a key on it. (Core concepts 6 and 10)

Common misconception

"My button does nothing, so my function must be wrong." A function can be perfect and never run. Before you change a line inside it, check three things, in this order.

First, the Console: a red `TypeError` about `null` means a name in block 1 holds no element, and the script stopped before your listener was connected. Second, block 3: with parentheses, as in `showNext()`, the function ran once at load and the listener got `undefined`. There is no red line, but the page has already changed with no click.

Third, the selector in block 1: it must match the id in the HTML, character by character. To test the function itself, call it by hand in the Console. If `showNext()` works there, the function is fine, and the problem is the connection. (Core concepts 3, 6 and 10; Project steps 10 and 16)

Exercise checks

These are the results with the lesson's code and its three sample interests.

- **Step 4.** Three list items. The first and the third have `favorite: true`, so they are bold and end with a star. The second has neither. (Core concept 9)

© 2026 559 Solutions LLC

- **Step 5.** Two `toggleDetails ran` lines after two clicks. After the first, the paragraph shows, the button says **Hide details** and turns dark blue. After the second, everything is back as it started. (Core concepts 6 to 8)
- **Step 6.** `3`. The loop logs three lines, such as `Interest one: a sentence about it`. The star is not in the text: an `<img>` has no text content. (Core concepts 4 and 5)
- **Step 7.** Four list items. The fourth has no star, because its `favorite` is `false`. The HTML did not change: the data decides how many items appear. (Core concept 9)
- **Step 8.** The first Tab from the address bar puts the ring on the button, the only thing on the page that can take focus. Enter shows the paragraph and Space hides it again, and each press adds one `toggleDetails ran` line. (Core concept 11)
- **Step 9.** The paragraph is already showing when the page loads, and the button says **Hide details** and is dark blue. Two clicks change nothing. The Console has exactly one `toggleDetails ran` line, from the load, and no red line. (Core concept 6)
- **Step 10.** The list stays empty and the button does nothing. The Console shows `Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')`, at `script.js:40:14`. Line 40 is `toggleButton.addEventListener(…)`. Without `defer`, the script ran when the browser reached it in the `<head>`, before the `<body>` existed. So `querySelector("#toggle-btn")` found nothing and gave `null`, and line 40 was the first line to use it. The script stopped there, so `buildList()` on line 41 never ran. (Core concept 2)

Self-assessment questions

1. **Why one mistake gives an error and the other does not.** In step 10, `nextButton.addEventListener("click", showNext())` is valid code. `showNext()` runs at once and returns `undefined`, and the browser accepts `undefined` as "no listener" (Core concept 6). Nothing fails, so nothing is printed. The clue is the page itself: it opened on photo 2 with no click, Next did nothing, and Previous still worked.

   In step 16, `querySelector("#question2")` gave `null`, and `question2.addEventListener(…)` reads a property of `null`. `null` has no properties at all, so the browser stops the script with a `TypeError`. The clue is the message: the line number and the name before `.addEventListener` lead back to the selector in block 1. Question 1 still worked, because its listener was connected before the error.

2. **A prediction you got wrong.** There is no single answer. A good answer quotes what you wrote down before you saved, says what the page or the Console really showed, and names the idea you understand now. For example: that `"Photo " + current + 1` joins the 1 as text, or that `showPhoto()` at the end of block 3 replaces the photo in the HTML.

3. **The friend's block 3.** Right after F5, the first answer is already open, with a `–` and a navy button, before any click. `toggleAnswer(question1, answer1, icon1)` has parentheses, so it ran once at load, and the listener got its return value, `undefined`. Each click on the first question then does nothing, and there is no red line.

© 2026 559 Solutions LLC

Questions 2 and 3 work, because their lines hand over a reference. A listener can only receive a function, not a call with its arguments. So keep `toggleAnswer` with its three parameters, and connect the small function that makes the call: `question1.addEventListener("click", toggleFirst);`. (Core concept 6; Project steps 10 and 14)

Project checks

- **Step 3.** The first photo, the caption and **Photo 1 of 5** show under the two buttons, and clicks do nothing. The Console has one red line, `Failed to load resource: net::ERR_FILE_NOT_FOUND`, for `gallery.js`.
- **Step 4, continue alone.** The hover rule:

```
.gallery-controls button:hover,
.faq-question:hover {
  background-color: var(--soft);
}
```

- **Step 5, continue alone.** The sample's other four photos are in the whole `gallery.js` at the end of this section. With them, `photos.length` gave `5`, `photos[1].alt` gave `A pot of soup on the stove`, and `current` gave `0`.
- **Step 6.** With `current = 2`, `showPhoto()` showed the third photo, its caption and **Photo 3 of 5**, and the Console printed `undefined`.
- **Step 7, continue alone.** The sample's `showPrevious`:

```
function showPrevious() {
  current--;
  if (current < 0) {
    current = photos.length - 1;
  }
  showPhoto();
}
```

  `current = 4`, then `showNext()`, showed **Photo 1 of 5**; then `showPrevious()` showed **Photo 5 of 5**.

- **Step 8.** Five clicks on Next go from photo 1 back to photo 1. One click on Previous from photo 1 shows **Photo 5 of 5**. Enter and Space each move one photo.
- **Step 9.** Without the parentheses: **Photo 01 of 5** at load, then **Photo 11 of 5**. With `let current = 4;`: **Photo 5 of 5** at load, then **Photo 1 of 5** after one click on Next.

© 2026 559 Solutions LLC

- **Step 10. Photo 2 of 5** at load, Next does nothing, Previous goes to **Photo 1 of 5**, and the Console has no red line.
- **Step 11, continue alone.** The sample's second and third questions, after the first `faq-item`:

```
<div class="faq-item">
  <h3>
    <button id="question-2" class="faq-question" type="button" aria-
expanded="false">
      <span id="icon-2" class="faq-icon">+</span>
      What do you do when you get stuck?
    </button>
  </h3>
  <p id="answer-2" class="faq-answer" hidden>I read the error from the top, then
check one thing at a time. A short walk helps too.</p>
</div>
<div class="faq-item">
  <h3>
    <button id="question-3" class="faq-question" type="button" aria-
expanded="false">
      <span id="icon-3" class="faq-icon">+</span>
      What do you do when you are not coding?
    </button>
  </h3>
  <p id="answer-3" class="faq-answer" hidden>I ride with a bike club every week.
Long routes are my favorite.</p>
</div>
```

- **Step 12, continue alone.**

```
.faq-icon {
  min-width: 24px;
  font-size: 24px;
  line-height: 1;
  text-align: center;
}

.faq-answer {
  margin: 0 0 16px;
  padding: 0 12px;
}
```

- **Steps 13 to 15, continue alone.** The six names for questions 2 and 3, the two small functions, and the two listeners are in the whole `faq.js` below. In step 13, `answer2.hidden` gave `true` and `icon3.textContent` gave `+`. In step 14, `toggleFirst()` opened the first answer,

© 2026 559 Solutions LLC

`question1.ariaExpanded` gave `true`, and a second call closed it. In step 15, with `hidden` deleted from `answer-1`, the icon and the color were the wrong way round after each click.

- **Step 16.** Question 1 works and questions 2 and 3 do not. The Console shows `Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')`, at `faq.js:42:11`.
- **Step 18.** If you followed the brief, every box is ticked. The most common misses: a leftover pair of parentheses from step 10, which only a click on Next shows, and an answer that lost its `hidden` in step 15.

Here are the sample's finished files. First, the whole `gallery.js`, 68 lines:

```
// gallery.js: one photo at a time on the Interests page, with Previous and Next

// 1. Find elements
const photo = document.querySelector("#gallery-photo");
const caption = document.querySelector("#gallery-caption");
const counter = document.querySelector("#photo-count");
const previousButton = document.querySelector("#prev-button");
const nextButton = document.querySelector("#next-button");

const photos = [
  {
    src: "images/interest-one-800.jpg",
    alt: "A trail through the hills",
    caption: "Hiking: the view at the top is worth the climb."
  },
  {
    src: "images/interest-two-800.jpg",
    alt: "A pot of soup on the stove",
    caption: "Cooking: chemistry I get to eat."
  },
  {
    src: "images/interest-three-800.jpg",
    alt: "Tomato plants in a raised bed",
    caption: "Gardening: from seed to salad."
  },
  {
    src: "images/interest-four-800.jpg",
    alt: "A guitar on a stand",
    caption: "Music: a little guitar every evening."
  },
  {
    src: "images/interest-five-800.jpg",
    alt: "A stack of library books",
    caption: "Reading: the best way to spend a rainy afternoon."
```


© 2026 559 Solutions LLC

```
  }
];

let current = 0;

// 2. Define behavior
function showPhoto() {
  const item = photos[current];
  photo.src = item.src;
  photo.alt = item.alt;
  caption.textContent = item.caption;
  counter.textContent = "Photo " + (current + 1) + " of " + photos.length;
}

function showNext() {
  current++;
  if (current === photos.length) {
    current = 0;
  }
  showPhoto();
}

function showPrevious() {
  current--;
  if (current < 0) {
    current = photos.length - 1;
  }
  showPhoto();
}

// 3. Connect events
previousButton.addEventListener("click", showPrevious);
nextButton.addEventListener("click", showNext);
showPhoto();
```

The whole `faq.js`, 43 lines:

```
// faq.js: questions about me on the About page; each button opens and closes its own
answer

// 1. Find elements
const question1 = document.querySelector("#question-1");
const answer1 = document.querySelector("#answer-1");
const icon1 = document.querySelector("#icon-1");

const question2 = document.querySelector("#question-2");
```


© 2026 559 Solutions LLC

```
const answer2 = document.querySelector("#answer-2");
const icon2 = document.querySelector("#icon-2");

const question3 = document.querySelector("#question-3");
const answer3 = document.querySelector("#answer-3");
const icon3 = document.querySelector("#icon-3");

// 2. Define behavior
function toggleAnswer(question, answer, icon) {
  answer.hidden = !answer.hidden;
  if (answer.hidden) {
    icon.textContent = "+";
  } else {
    icon.textContent = "−";
  }
  question.classList.toggle("is-open");
  question.ariaExpanded = !answer.hidden;
}

function toggleFirst() {
  toggleAnswer(question1, answer1, icon1);
}

function toggleSecond() {
  toggleAnswer(question2, answer2, icon2);
}

function toggleThird() {
  toggleAnswer(question3, answer3, icon3);
}

// 3. Connect events
question1.addEventListener("click", toggleFirst);
question2.addEventListener("click", toggleSecond);
question3.addEventListener("click", toggleThird);
```

The two new groups at the end of `styles.css`, above the wider-screens block, are the rules from Project steps 4 and 12 with the two "Continue alone" parts above filled in. In `interests.html` and `about.html`, the only changes are the `<script>` line in the `<head>` and the new section before `</main>`.

NEXT

Module 10: Forms and Validation with JavaScript.
Free lesson: 559solutions.com/code559/module-10-forms-validation/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC